HybridForge RAG

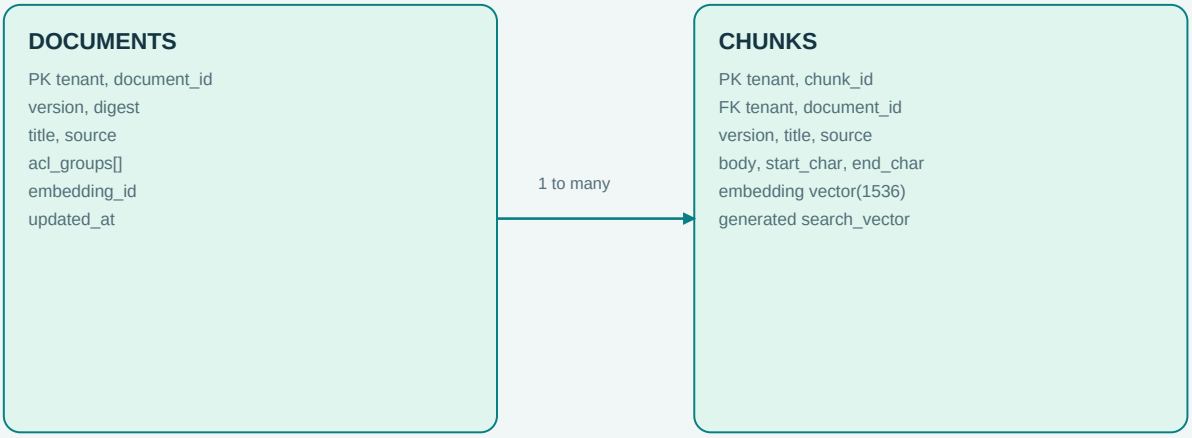
Architecture, implementation and production qualification | Version 1.0 | 16 October 2026

Answers with an evidence trail.

A runnable hybrid retrieval system that combines lexical search and dense similarity, enforces tenant permissions, and returns structured answers with inspectable source citations.

Tenant-scoped relational model

A document and its lexical/vector chunks commit together



Foreign key cascade deletes active chunks. Both content tables FORCE row-level security.

DELIVERED	QUALIFICATION
Source, API and browser workbench	38 automated tests passed; 3 PostgreSQL tests skipped
Real-provider and database adapters	Paid model calls and production database behavior not executed here
Diagrams, runbooks and deployment templates	Not deployed; production acceptance remains open

Prepared as an engineering project and review package. Local evidence supports the verified features only; it is not a production certification.

System at a glance

Read the Markdown documents for detailed contracts, decisions and operational procedures.

AREA	WHAT IS IMPLEMENTED
Retrieval	PostgreSQL full-text + exact pgvector search, RRF fusion and optional reranker adapter
Data lifecycle	Document fingerprints, optimistic version checks, atomic replace and cascading delete
Security	OIDC RS256 verifier, scopes, group ACLs, forced database row-level security
Generation	Real embedding/chat API adapter, structured claims, source-ID and quote validation
Experience	Same-origin browser workbench, evidence explorer, source versions and diagnostic modes
Operations	Input bounds, local admission controls, Prometheus metrics and optional OTLP traces

Two deliberately different modes

Offline demo: SQLite, Python BM25, deterministic hash vectors and evidence extraction. Runs without model credentials. Hash vectors do not provide learned semantic search.

Production adapter: PostgreSQL/pgvector, real embeddings and structured model generation. Requires verified identity, database configuration, provider credentials and deployment qualification.

Guide map

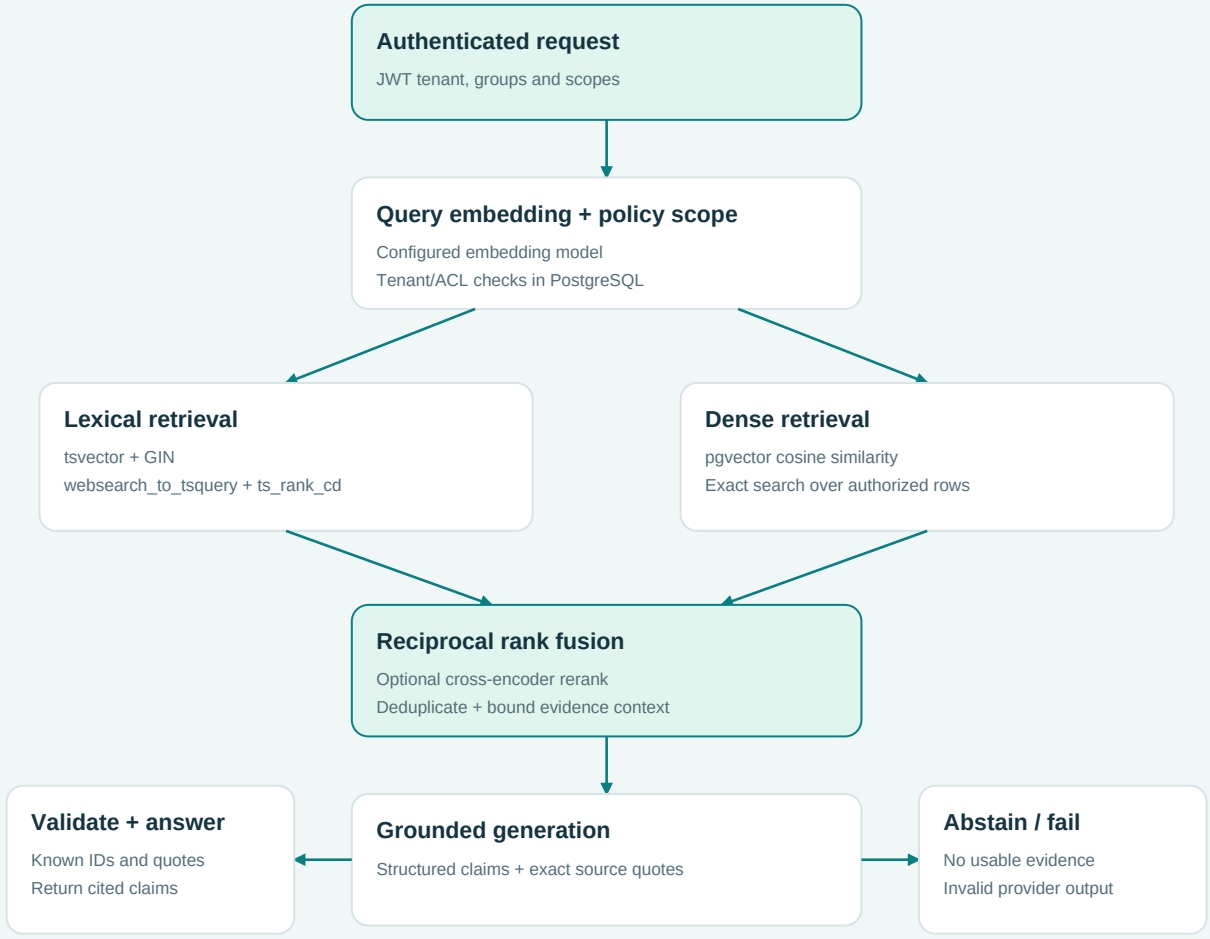
03 architecture; 04 retrieval mechanics; 05 ingestion; 06 security; 07 deployment; 08 API; 09 evaluation; 10 operations; 11 release gates; 12 decisions and references.

Architecture

Policy scope is established before evidence can reach rank fusion or the model.

Retrieval with an explicit evidence trail

Production path: PostgreSQL + learned embeddings + structured generation



Security scope applies before candidate fusion and before sending text to the model.

Quote validation checks provenance. It does not prove that a claim follows from the quote.

Lexical and dense SQL searches share one repeatable-read transaction. Generation occurs after the transaction, using the selected evidence snapshot. Permission changes after retrieval do not recall evidence already in flight.

How hybrid retrieval works

Separate retrieval signals; one traceable, bounded evidence set.

STAGE	MECHANISM	WHY IT MATTERS
Lexical	tsvector, GIN and ts_rank_cd	Handles exact identifiers and vocabulary matches
Dense	1 minus cosine distance	Finds learned conceptual similarity in real-provider mode
Fusion	Sum of 1 / (60 + rank)	Combines different ranking scales without adding raw scores
Reranking	Optional cross-encoder	Reorders fused candidates using query-passage relevance
Selection	Overlap suppression and character budget	Avoids wasting context on duplicate passages
Generation	Structured claims with exact quotes	Produces a provenance-checkable response contract

A concrete fusion example

If chunk A ranks first lexically and fourth semantically, its fused score is 1/61 + 1/64 = 0.03202. A chunk seen only at dense rank one scores 1/61 = 0.01639. These values rank evidence; they are not probabilities that an answer is correct.

Intentional v1 limits

The PostgreSQL dense branch performs an exact scan over the authorized subset. This establishes a correctness baseline but needs a measured corpus limit. Simply creating HNSW will not accelerate the current materialized-subset query.

PostgreSQL full-text ranking is not BM25. Long natural-language queries may be too restrictive under websearch_to_tsquery; evaluate normalization or an alternate lexical backend against your data.

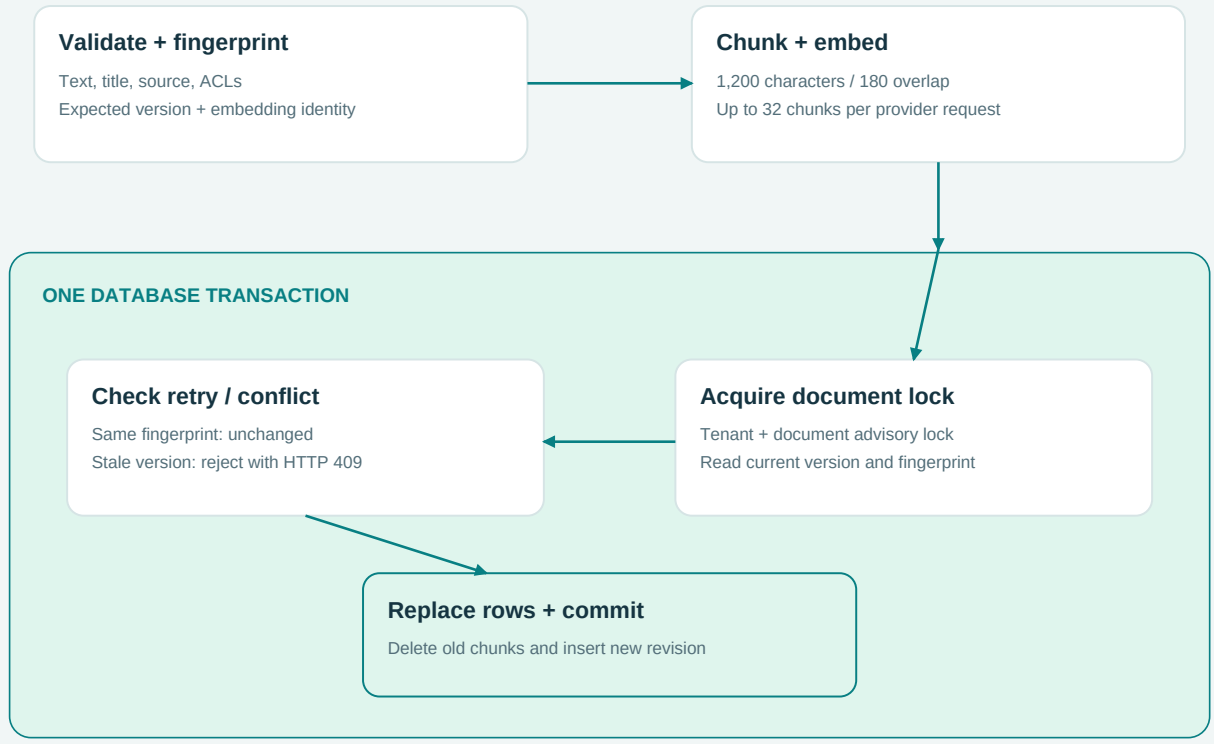
Default bounds: 30 candidates per branch, at most 100; top_k defaults to 5 and is capped at 12; evidence budget is 16,000 characters. A 0.25 dense threshold is a starting parameter, not a calibrated confidence threshold.

Ingestion without partial updates

Create, retry, replace and delete have explicit transactional semantics.

Atomic document replacement

Embeddings are prepared before opening the database write transaction



Failure before commit: the prior committed document remains available.

Active revisions are versioned; historical source bodies are not archived in v1.

The API accepts text/Markdown in JSON. It does not crawl URLs, parse PDFs or execute external connectors. A durable ingestion queue is an extension for workloads beyond the bounded synchronous path.

Security and answer trust

Protect the evidence path first; treat generated claims as untrusted output.

BOUNDARY	CONTROL	REMAINING RESPONSIBILITY
Identity	RS256, issuer, audience, expiry and required claims	Correct claim issuance and short token lifetime
Tenant / ACL	Forced RLS plus explicit tenant filters	Trusted service credentials and least-privilege networking
Model input	Only selected authorized chunks	Provider retention, region and approved data handling
Model output	Known chunk IDs and exact quote membership	Semantic entailment and source truth evaluation
Browser	textContent rendering and self-only CSP	Review future rich-text or link features
Resource use	Body, context, candidate and concurrency bounds	Gateway deadlines and fleet-wide quotas

A valid quote is not proof of a valid claim

The validator rejects an invented source ID or a quote that is absent from its cited chunk. It cannot prove that a claim follows from a valid quote. A malicious or mistaken document can still produce misleading output. Add independent answer-quality review and adversarial testing before deployment.

Permission and deletion semantics

Empty ACL means tenant-wide reading. Writer scope grants tenant administration. Token group changes take effect when new tokens are issued; in-flight requests use a retrieval snapshot. Deletion removes active rows, not already returned answers, backups or provider-retained copies.

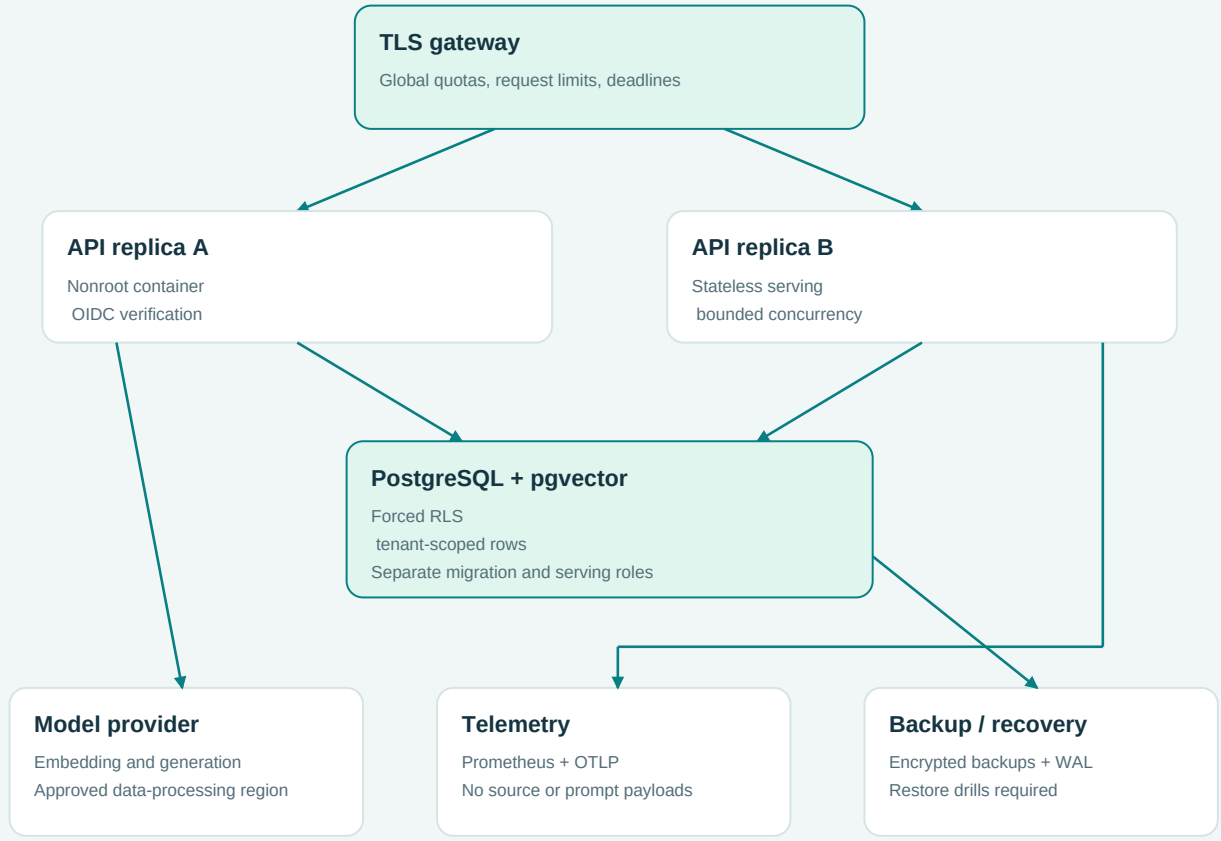
RLS uses transaction-local context. A compromised service database credential remains a serious breach: trusted application code can set the policy context. RLS is defense in depth, not a sandbox for arbitrary SQL access.

Deployment topology

Separate application behavior from the infrastructure required to operate it.

Production topology and ownership

Application templates are delivered; hosted infrastructure is operator-provisioned



Identity, ingress, secrets, database hosting and observability are external dependencies.

The supplied manifests have not been deployed or load-tested in this delivery.

Kubernetes templates include two replicas, readiness/liveness probes, resource limits, a disruption budget, nonroot execution and an example network policy. Operators must customize and validate them in their own cluster.

API and integration contract

All content endpoints use a verified bearer principal; the body cannot choose a tenant.

ENDPOINT	SCOPE	RESULT
PUT /v1/documents	rag:write	Create or replace with expected_version
GET /v1/documents	rag:read	Visible metadata; PostgreSQL list capped at 1,000
DELETE /v1/documents/{id}	rag:write	Delete matching version and its chunks
POST /v1/search	rag:read	Selected evidence and rank diagnostics
POST /v1/query	rag:read	Cited claims or explicit abstention
GET /metrics	rag:observe	Per-process Prometheus metrics
GET /health/ready	Public	Database readiness; no provider probe

Document preconditions

Create with expected_version=0. For changed content, use the current version. Repeated identical fields are idempotent and preserve that version. A stale changed update returns 409. Text is limited to 100,000 characters, the full body to 700,000 bytes and generated chunks to 128.

Errors and retries

401/403 require valid credentials or permissions; 409 requires refreshing the document version; 413/422 require payload changes. Retry 429/502/503 with bounded backoff. Provider retries may incur additional charges. Invalid citations reject the whole generated answer rather than silently returning a partial result.

The answer contains document ID, revision, source label, exact quote and chunk character range. Character offsets identify the full chunk, not just the quoted substring. See docs/API.md and docs/openapi.json.

Evaluation and evidence

Measure retrieval, authorization and generation independently.

OFFLINE MODE	RECALL@5	MRR	nDCG@5
lexical	1.00	1.00	1.00
dense	0.40	0.40	0.40
hybrid	1.00	1.00	1.00

Actual local run: ten synthetic questions against six fictional documents. Dense mode uses hash vectors, not semantic embeddings. This result demonstrates the wiring and lexical recovery; it does not establish that hybrid outperforms a real semantic model.

Verified test evidence

38 automated tests passed and three PostgreSQL integration tests skipped. Coverage across the Python package was 84%, including unexecuted database/reranker/telemetry paths. A Python wheel was built successfully. Detailed evidence and environment limits are in docs/VALIDATION.md.

Production evaluation plan

Use a held-out corpus with exact codes, paraphrases, ambiguous questions, multi-document answers, unanswerable queries, conflicting revisions and injected instructions. Split by source family and time. Compare lexical, dense, hybrid and reranked hybrid under the same ACLs and budgets.

Require zero unauthorized chunks. Measure answer faithfulness, completeness, refusal quality, quote correctness and semantic entailment separately. Qualify p50/p95/p99 latency, failure recovery and cost at the target tenant/corpus scale. A small synthetic recall score is not a production quality gate.

Operate, observe, recover

Release and incident workflows are part of the engineering deliverable.

Deployment sequence

Provision PostgreSQL and backups; run the migration with a separate owner identity; configure rag_app, OIDC and model credentials; deploy a scanned image behind TLS; verify isolation and real-provider smoke tests; then canary and expand after acceptance gates pass.

Observability

Prometheus tracks operation counts by bounded outcome labels and duration histograms. Optional OTLP spans cover embedding, storage, retrieval, reranking and generation. Application logs contain operation, outcome and request ID; source text, prompts and credentials are excluded. Protect metrics with rag:observe and network controls.

SYMPTOM	FIRST RESPONSE
401 / 403 spike	Inspect issuer, audience, scopes, group mapping and JWKS rotation; never disable validation
502 provider failures	Check quota, availability and structured-output compatibility; preserve fail-closed behavior
Slow queries / 503	Check admission limits, pool capacity, authorized corpus size and provider latency
Empty evidence	Check tenant/ACLs, model identity, ingestion completeness and lexical semantics
Suspected disclosure	Stop exposure, revoke credentials and preserve restricted incident evidence

Recovery and cost

Use encrypted backups with WAL archiving and perform isolated restore drills. Verify schema, extension, role grants, forced RLS and tenant-negative tests after restoration. Budget embedding overlap, reingestion and retries. Global cost controls and hard workflow deadlines are not implemented.

Production acceptance gates

Current status: runnable reference implementation, not production-certified.

GATE	REQUIRED EVIDENCE
Database correctness	Execute all PostgreSQL integration and RLS tests using the actual non-owner role
Live provider compatibility	Verify embeddings, structured generation, real JWKS and key rotation
Quality and safety	Held-out retrieval metrics, expert answer review and injection/entailment tests
Capacity and resilience	Load/soak results, service limits, provider outage drills and DB recovery
Supply chain	Image build/scan, SBOM, dependency review, base-image/action/model pins
Infrastructure	TLS and CA validation, secret rotation, alert delivery and backup restore
Release process	Staging acceptance, canary, rollback rehearsal and named owners

Explicitly outside the implemented scope

PDF/OCR ingestion, crawlers, external source connectors, durable asynchronous ingestion, immutable revision storage, conversation memory, answer caching, distributed rate limits and ANN query execution are not implemented. Add them only with the corresponding authorization, retention and failure semantics.

The optional cross-encoder adapter exists but was not executed here. Docker, Kubernetes, hosted database operation and paid model calls remain unverified. Do not present the deployment templates or local smoke scores as evidence of production scale.

Decisions and primary references

Trace design choices to documentation and retain an auditable qualification record.

DECISION	RATIONALE
Single PostgreSQL store	Keep ACL, document and retrieval representations transactional
RRF ranking	Avoid mixing uncalibrated raw lexical and vector scores
Exact authorized dense scan	Establish a clear permission-aware retrieval baseline
Structured claims + quotes	Make source provenance deterministically checkable
Explicit offline mode	Allow no-key demos without pretending hash vectors are semantic
No cache or memory in v1	Avoid stale entitlement and conversation-retention complexity

Primary technical references

pgvector
https://github.com/pgvector/pgvector

PostgreSQL row security
https://www.postgresql.org/docs/current/ddl-rowsecurity.html

PostgreSQL full-text search
https://www.postgresql.org/docs/current/textsearch-controls.html

OpenAI embeddings guide
https://developers.openai.com/api/docs/guides/embeddings

Document set

The project includes ARCHITECTURE, API, SECURITY, OPERATIONS, EVALUATION, DECISIONS, PRODUCTION_READINESS, VALIDATION and REFERENCES Markdown documents; editable Mermaid and SVG diagrams; API schema; dependency locks; test reports; and deployment configuration. Consult those files for full implementation details.